

Introduction to Machine Learning Methods

Gradient descent for linear regression

Haris Gačanin
RWTH Aachen University

Outline

- Recap
 - Heuristic search
 - Gradient search
 - The weight space and hypothesis
 - Locally weighted regression
 - Probabilistic interpretation of linear regression
-
- Reference chapters: 18.6.1-18.6.3, 18.8.4

Recap: Linear function

- We are all familiar with this linear model

$$\begin{aligned}y(x) &= ax + b \\ &= b + ax\end{aligned}$$

x = one independent variable

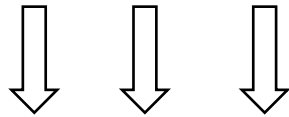
y = output variable

a and b are parameters of a function

This is called univariate linear regression because there is only one predictor variable/feature

The hypothesis is the same linear function

$$y(x) = b + ax$$



$$h(x) = \theta_0 + \theta_1 x$$

Recap: Search for the “best” line fit (univariate model)

- Select θ_0 and θ_1 so that $h_{\theta}(x^{(i)}) \approx y^{(i)}$ for m training examples $(x^{(i)}, y^{(i)})$

$$\begin{aligned}\min_{\theta} J(\theta) &= \arg \min_{\theta_0, \theta_1} \frac{1}{2m} \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}]^2 \\ &= \arg \min_{\theta_0, \theta_1} \frac{1}{2m} \sum_{i=1}^m [\theta_0 + \theta_1 x^{(i)} - y^{(i)}]^2\end{aligned}$$

- **Note:** Introducing a $\frac{1}{2}$ in the coefficient i.e., $\frac{1}{2m}$ is purely for computational convenience in gradient-based optimization

Recap: Search for the “best” line fit (bivariate model)

- Select θ_0 , θ_1 and θ_2 so that $h_{\theta}(x^{(i)}) \approx y^{(i)}$ for m training examples $(x^{(i)}, y^{(i)})$

$$\min_{\theta} J(\theta) = \arg \min_{\theta_0, \theta_1, \theta_2} \frac{1}{2m} \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}]^2$$

$$= \arg \min_{\theta_0, \theta_1, \theta_2} \frac{1}{2m} \sum_{i=1}^m [\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - y^{(i)}]^2$$

- **Note:** Introducing a $\frac{1}{2}$ in the coefficient i.e., $\frac{1}{2m}$ is purely for computational convenience in gradient-based optimization

-
- Select $\theta_0, \theta_1 \dots \theta_n$ so that $h_{\theta}(x^{(i)}) \approx y^{(i)}$ for m training examples $(x^{(i)}, y^{(i)})$

$$\begin{aligned}\min_{\theta} J(\theta) &= \arg \min_{\theta_0, \theta_1, \dots, \theta_n} \frac{1}{2m} \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}]^2 \\ &= \arg \min_{\theta_0, \theta_1, \dots, \theta_n} \frac{1}{2m} \sum_{i=1}^m [\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots + \theta_n x_n^{(i)} - y^{(i)}]^2\end{aligned}$$

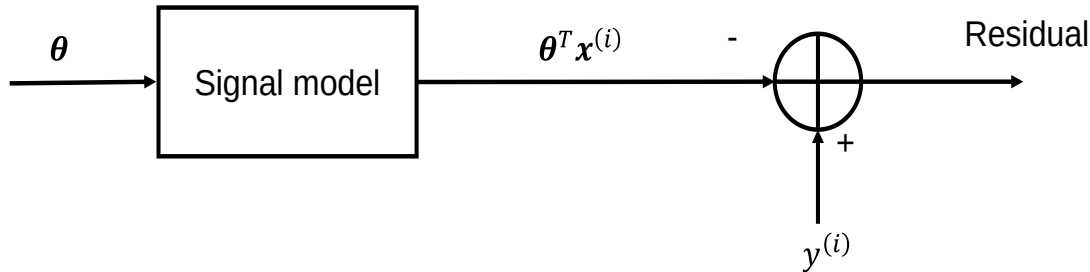
- **Note:** Introducing a $\frac{1}{2}$ in the coefficient i.e., $\frac{1}{2m}$ is purely for computational convenience in gradient-based optimization

Recap: LS method

- The **least-squares method** searches for the best fit of a data points by minimizing the sum of the squared residuals

$$\sum_{i=1}^m (y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2 = \sum_{i=1}^m \left(y^{(i)} - \sum_{k=0}^1 \theta_k x_k^{(i)} \right)^2 = \sum_{i=1}^m (y^{(i)} - \theta_0 + \theta_1 x_1)^2$$

Q: What are our options to update parameters?



Recap: Can a cost function $J(\theta)$ have a value of 0?

- Under which condition

$$J(\theta) = 0$$

Recap: Can a cost function $J(\theta)$ have a value of 0?

- Under which condition

$$J(\theta) = 0$$

- The cost function will have zero value when

$$h_{\theta}(x^{(i)}) - y^{(i)} = 0 \quad \forall i$$

- What does this mean?

Recap: Can a cost function $J(\theta)$ have a value of 0?

- Under which condition

$$J(\theta) = 0$$

- The cost function will have zero value when

$$h_{\theta}(x^{(i)}) - y^{(i)} = 0 \quad \forall i$$

- What does this mean?
 - Every prediction is exactly matched to actual function!
 - Most probably leads to overfitting

Remarks

- A cost of 0 is possible mathematically, but not realistic in real-world datasets because of:
 - noise, wrong labels, regularization terms (we see later about this)
- Some cost functions are always nonnegative:

$$J(\theta) \geq 0$$

because they are based on squares, absolute values, or probabilities.

- **Note:** for gradient descent, the goal is to minimize the cost toward a minimum value, which may or may not be exactly 0!

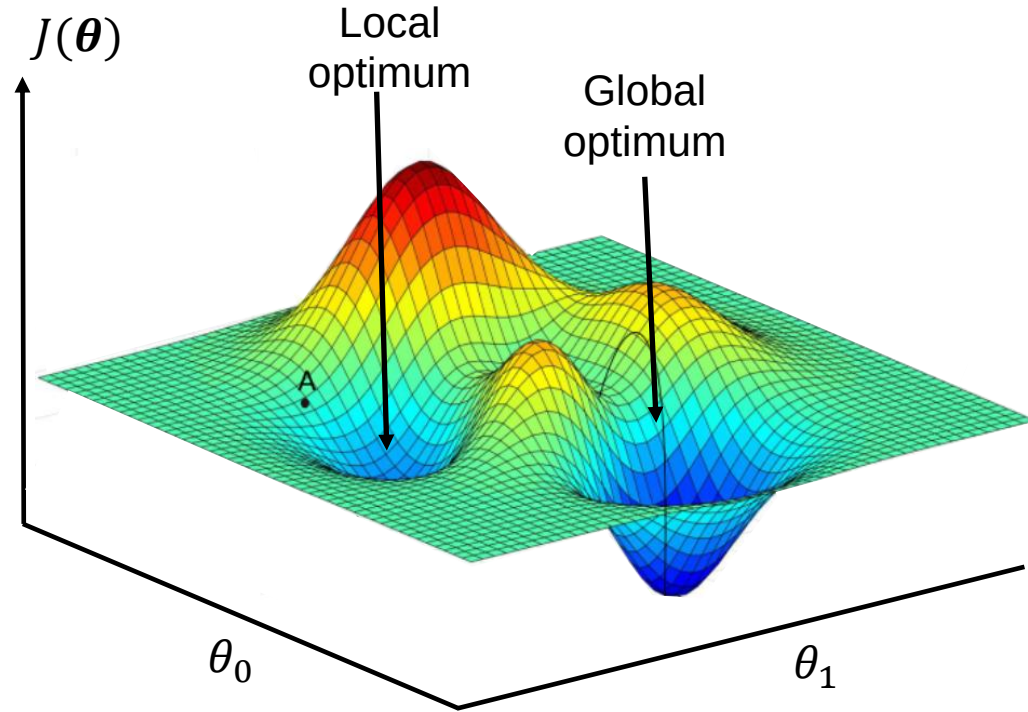


Conceptualization of Heuristic search and cost function

Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

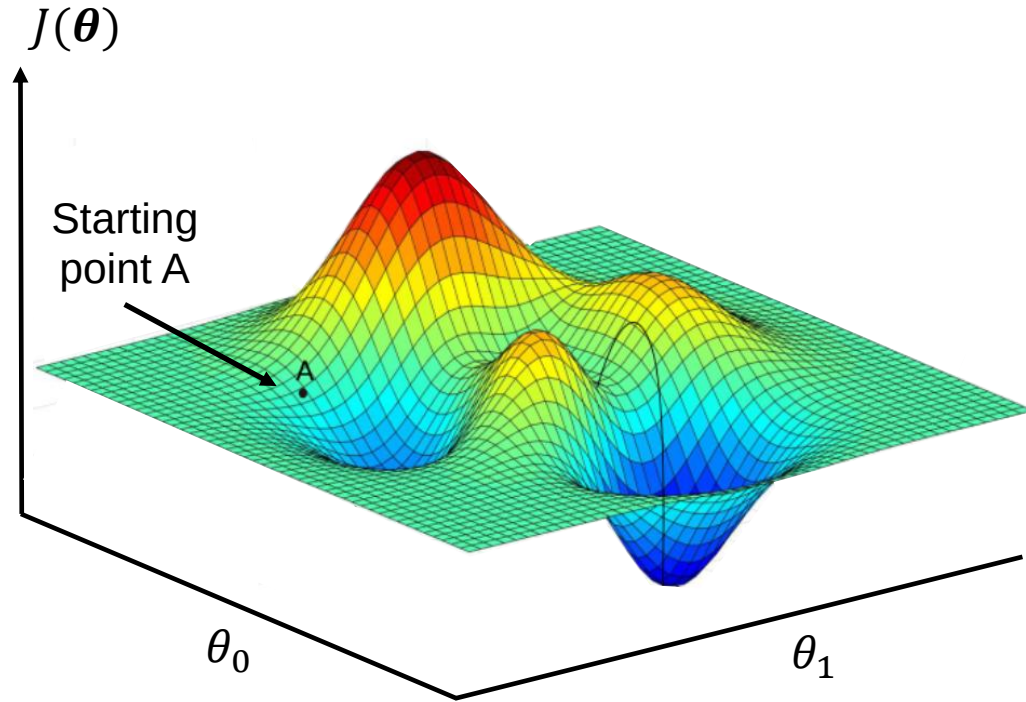


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$

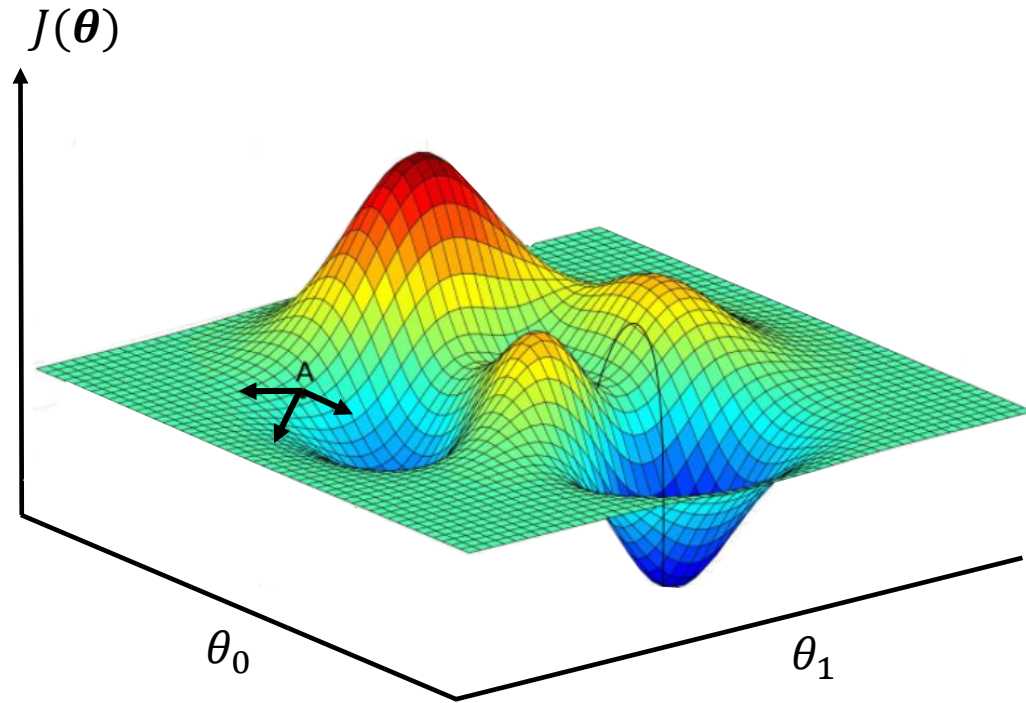


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$

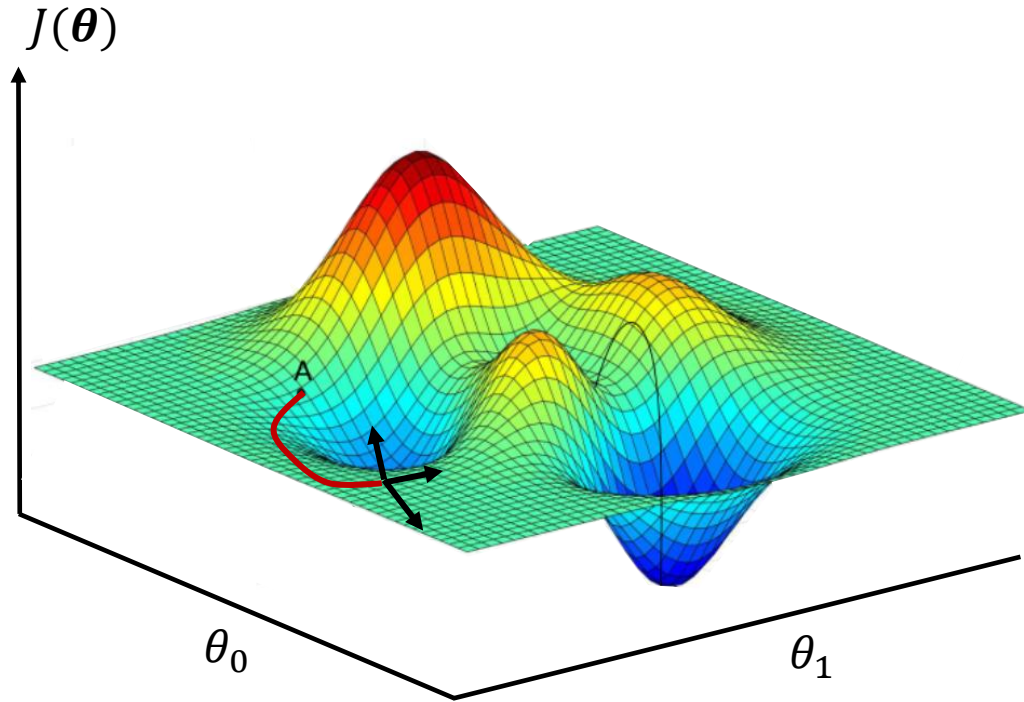


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$
- Walk through to minimize
 $J(\theta_0, \theta_1)$

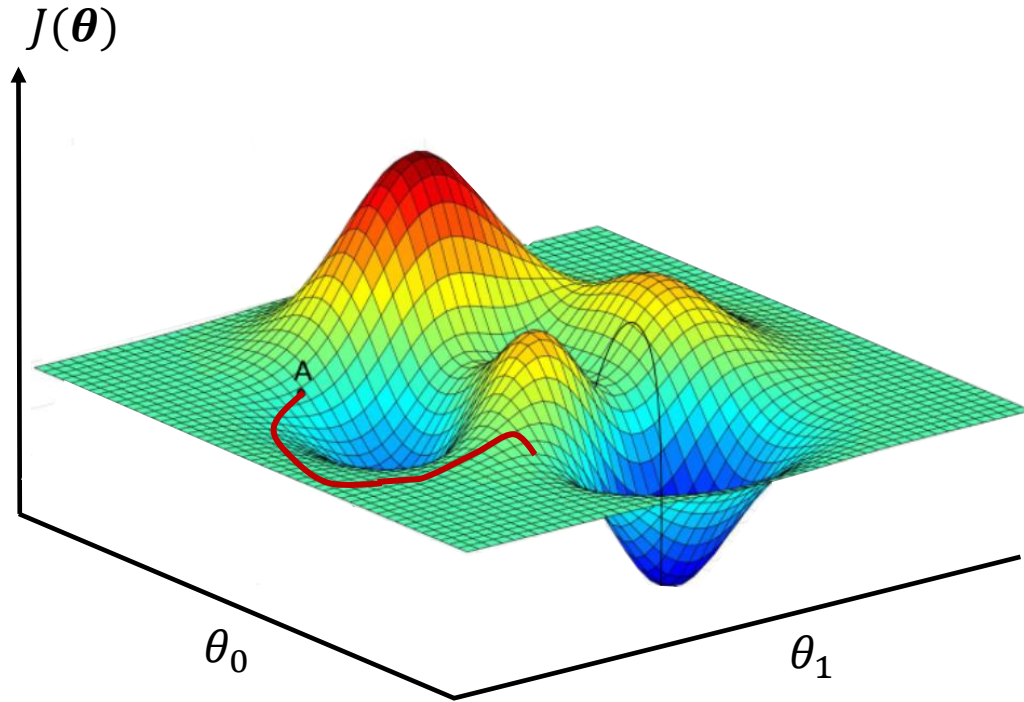


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$
- Walk through to minimize
 $J(\theta_0, \theta_1)$

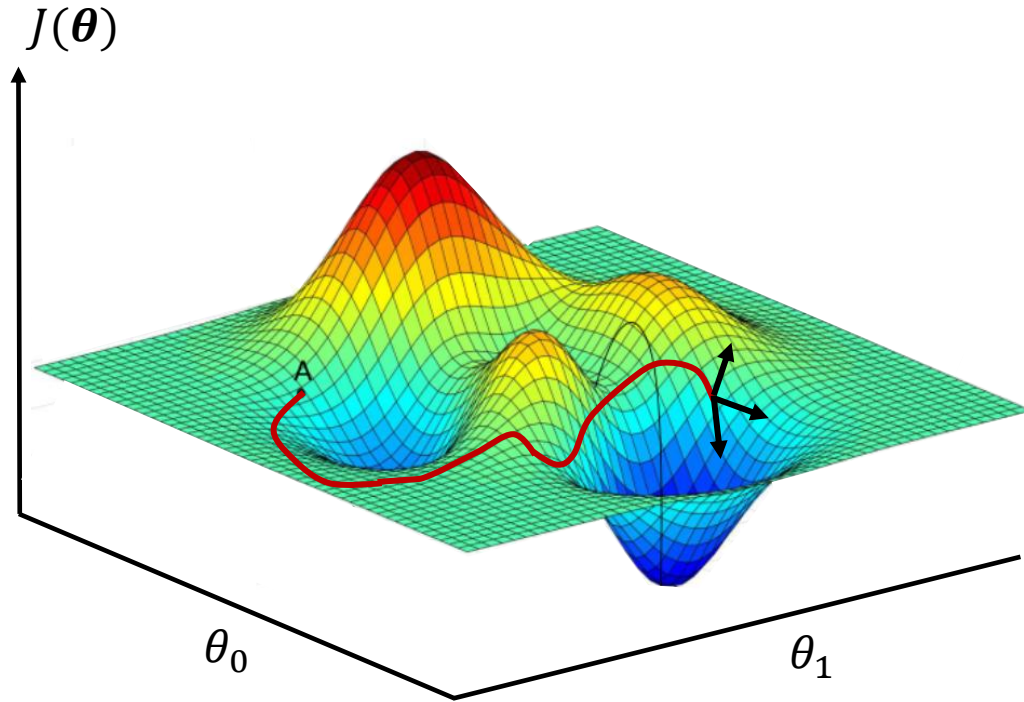


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$
- Walk through to minimize
 $J(\theta_0, \theta_1)$

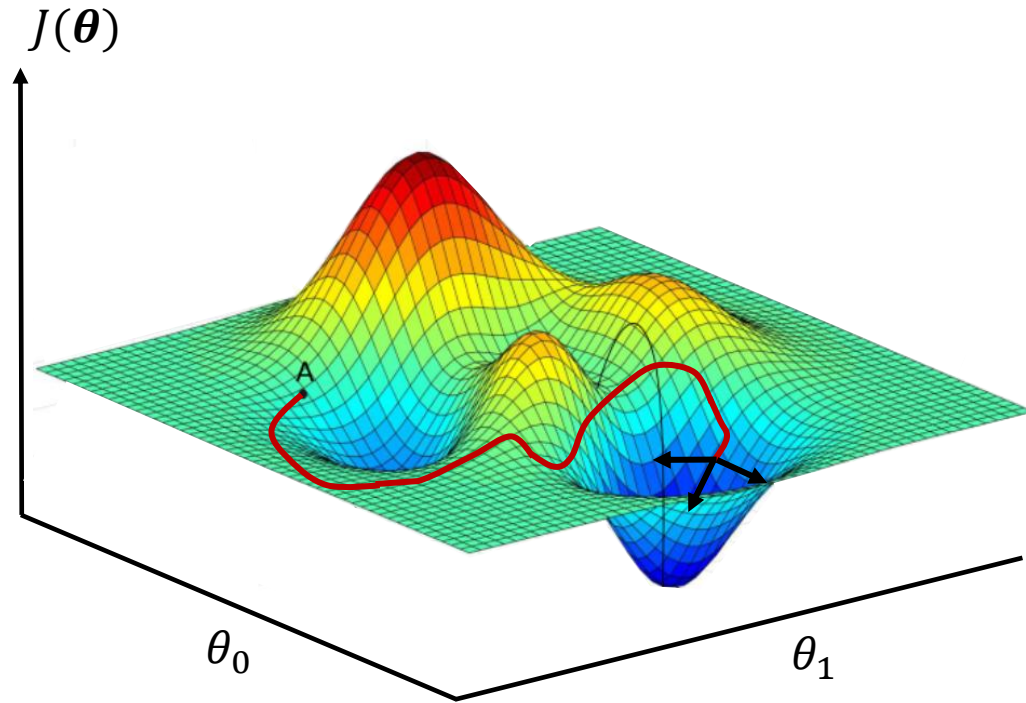


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$
- Walk through to minimize
 $J(\theta_0, \theta_1)$

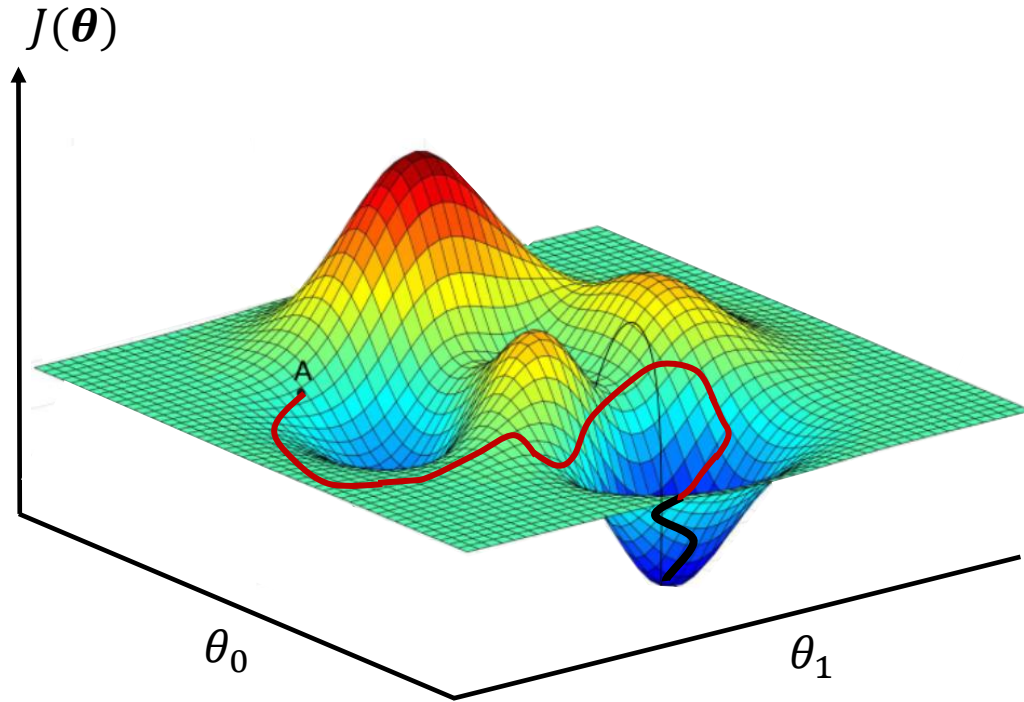


Search heuristic solution (univariate example)

$$\text{Goal: } \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Heuristic:

- Select a random starting point
 $A = \{\theta_0, \theta_1\}$
- Update $\{\theta_0, \theta_1\}$ and observe
 $J(\theta_0, \theta_1)$
- Walk through to minimize
 $J(\theta_0, \theta_1)$

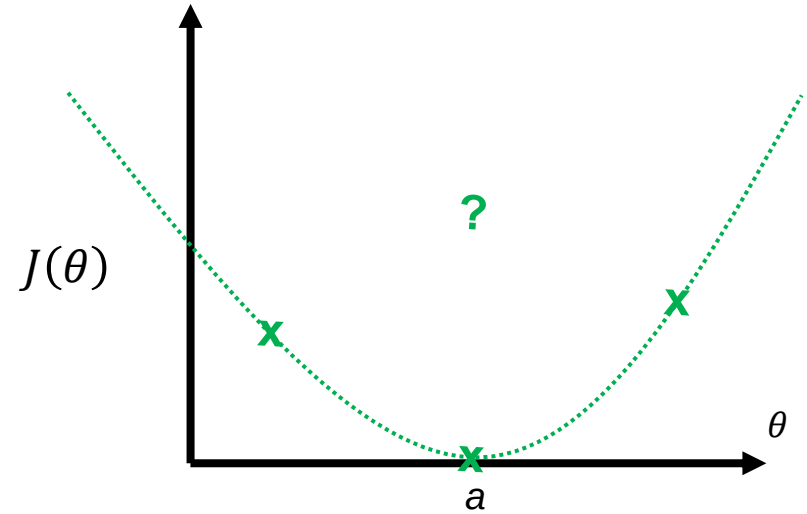
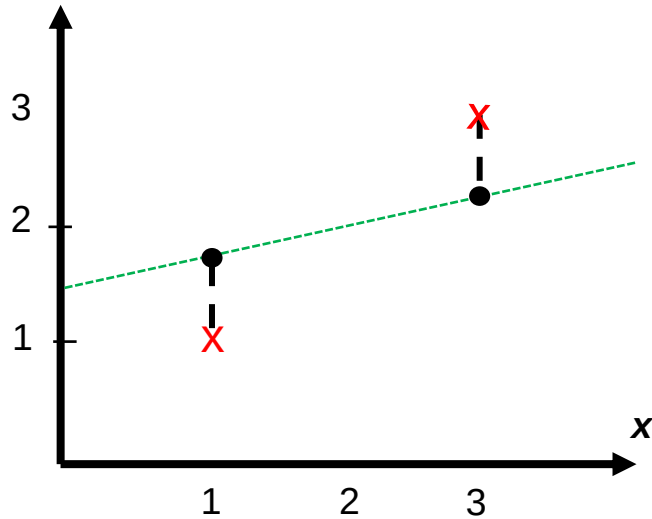




Relationship of a linear model and a cost function

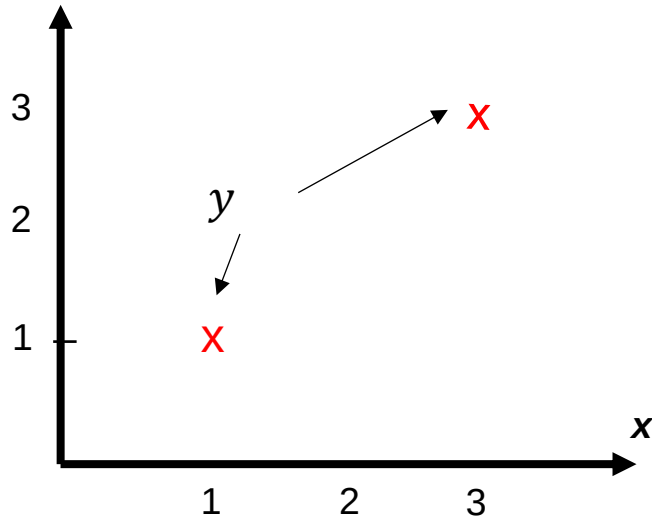
How to use intuition?

$$h_{\theta}(x) = ? + ? x$$



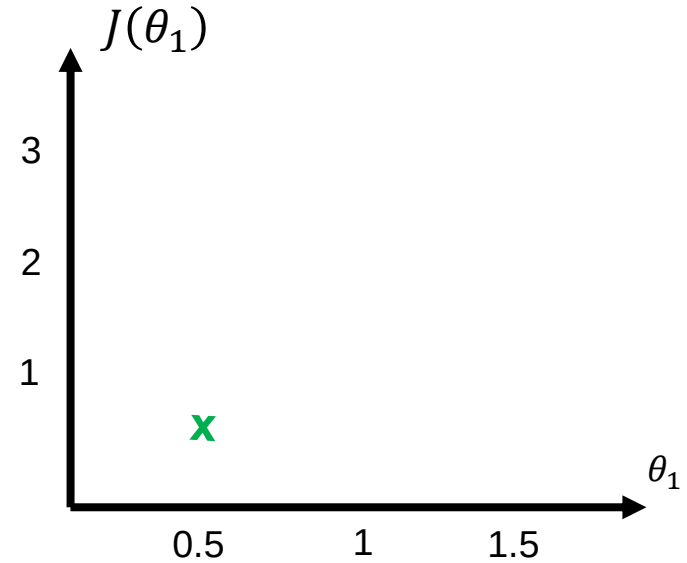
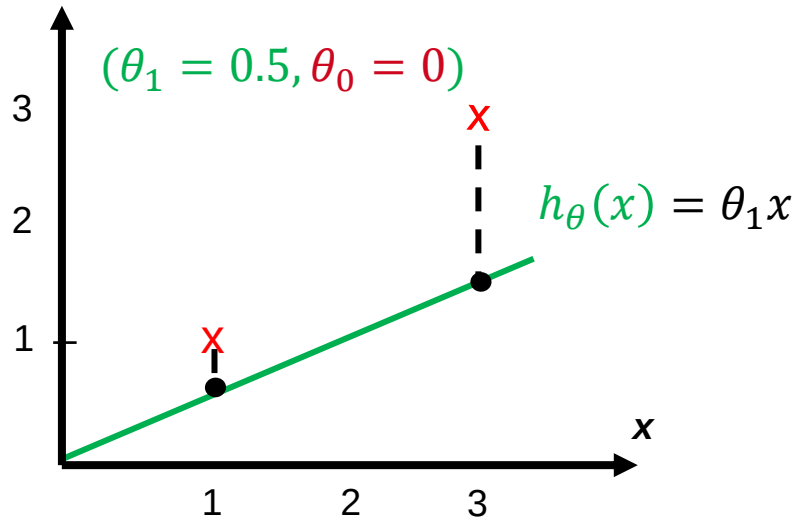
Relationship between a model and loss function

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



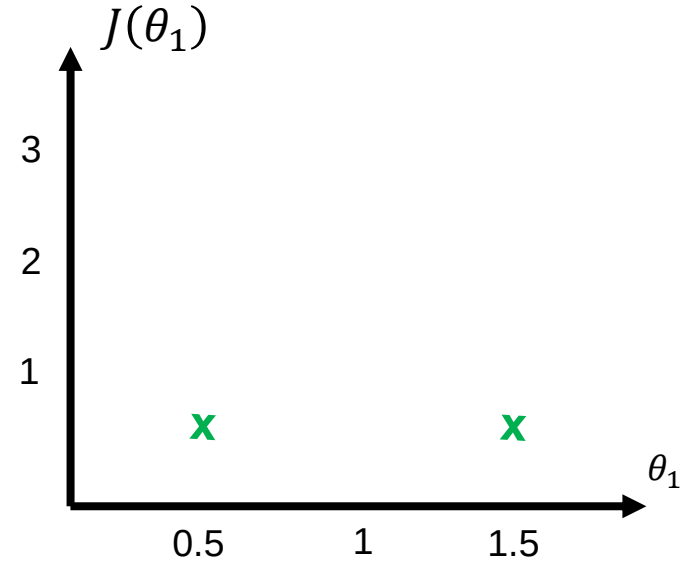
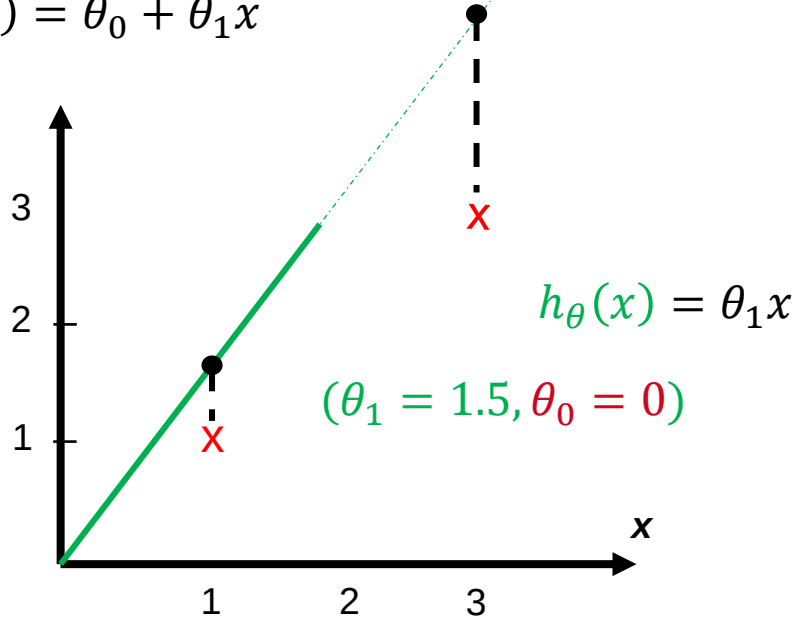
Relationship between a model and loss function

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



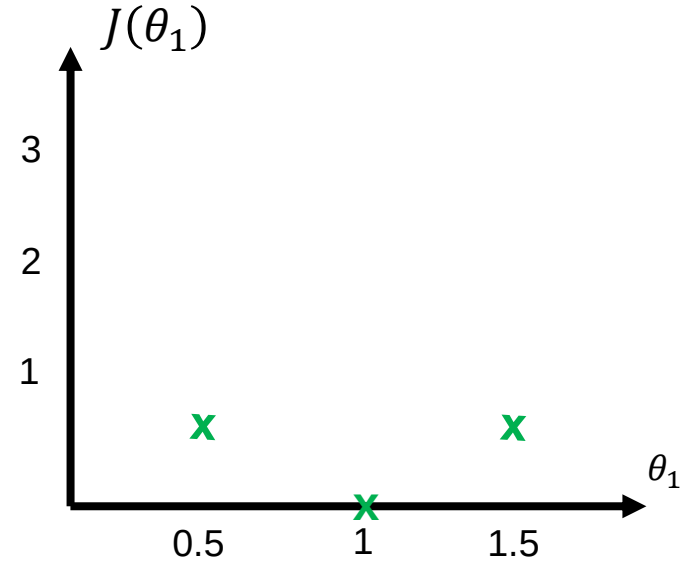
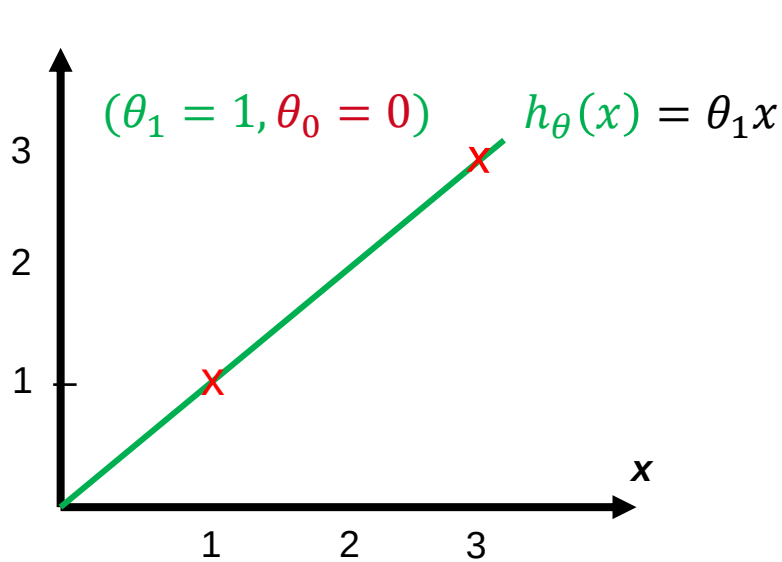
Relationship between a model and loss function

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



Model and loss function

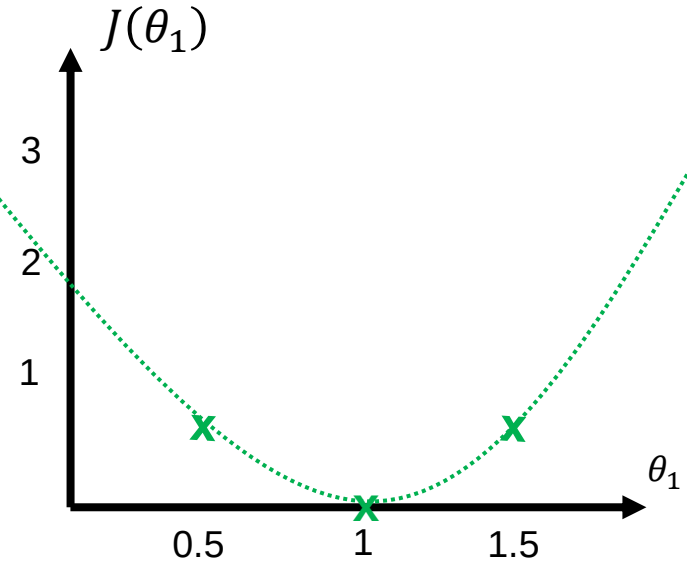
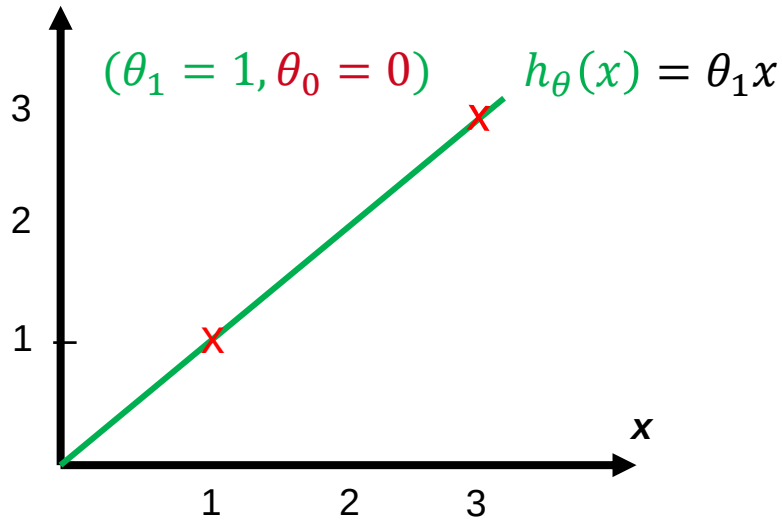
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



A model is matching actual values, so $J(\theta_1) = 0$

Model and loss function

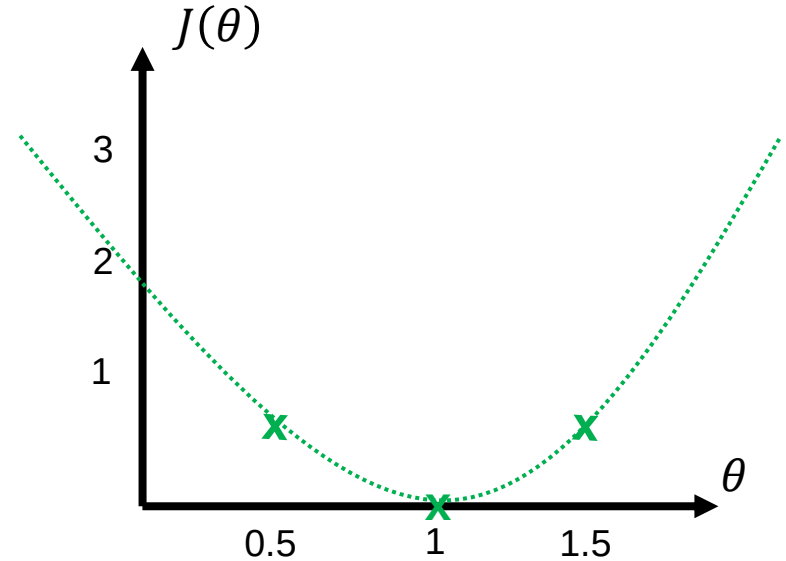
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



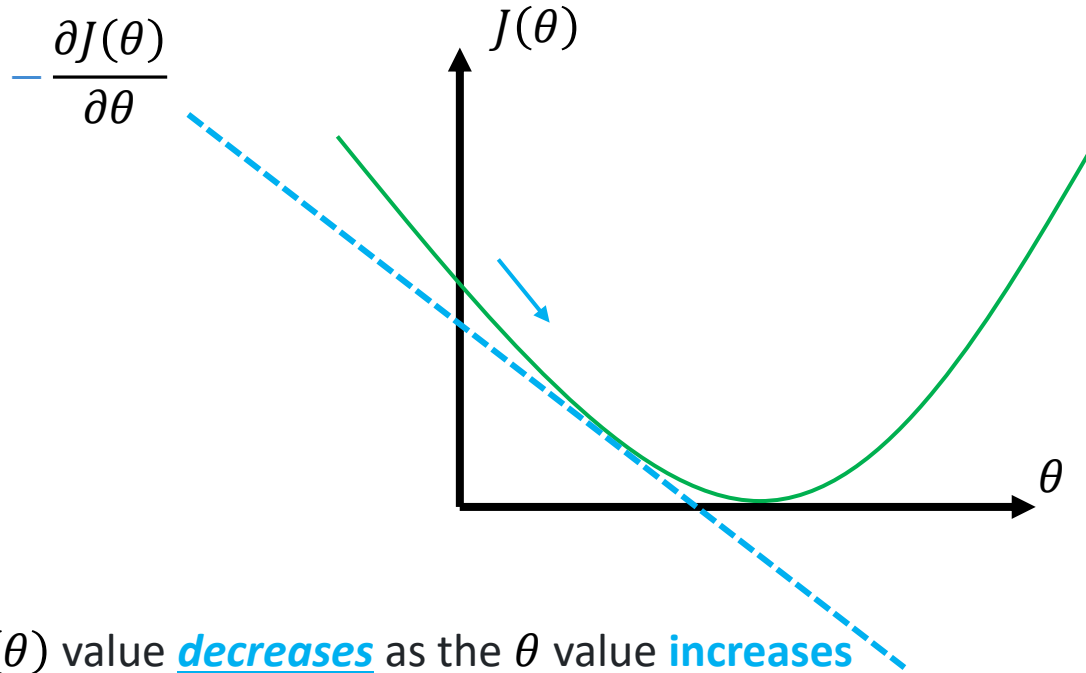
Minimum point of a function

- The minimum point of $J(\theta)$ is determined by a gradient

$$\frac{\partial J(\theta)}{\partial \theta}$$

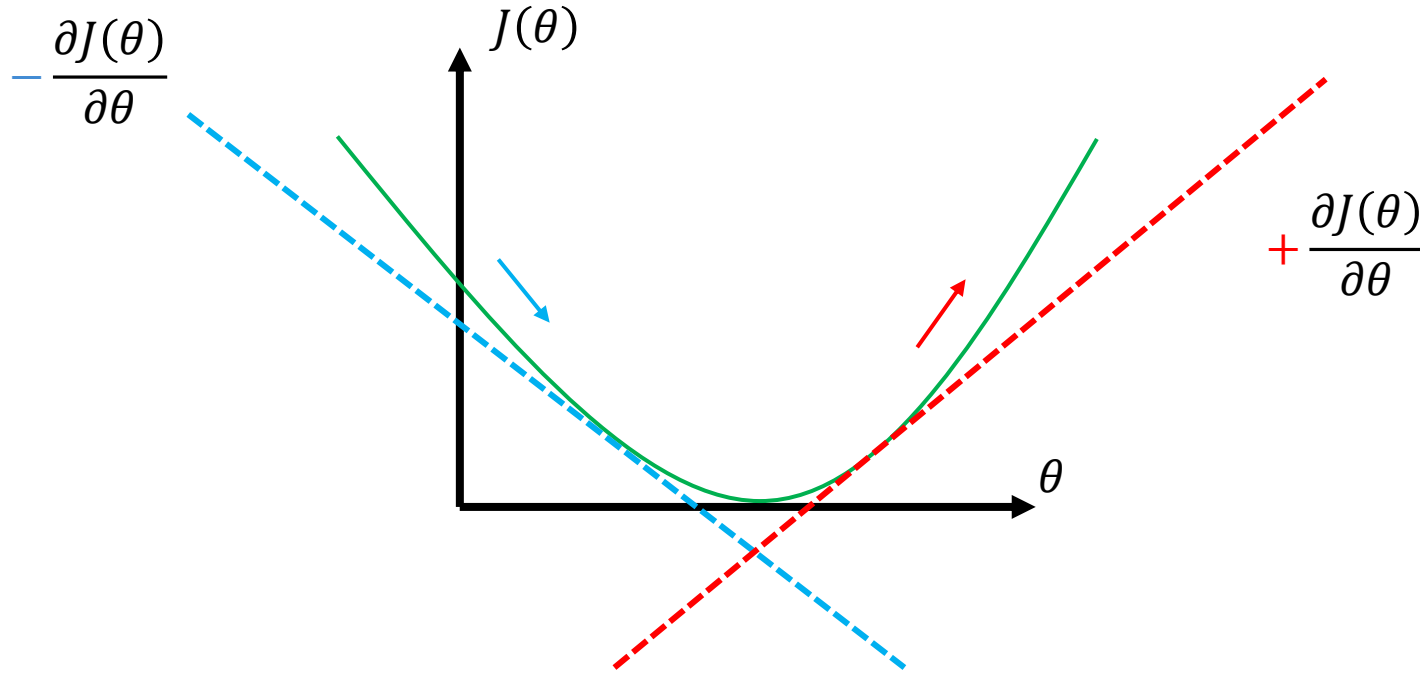


Where points a gradient?

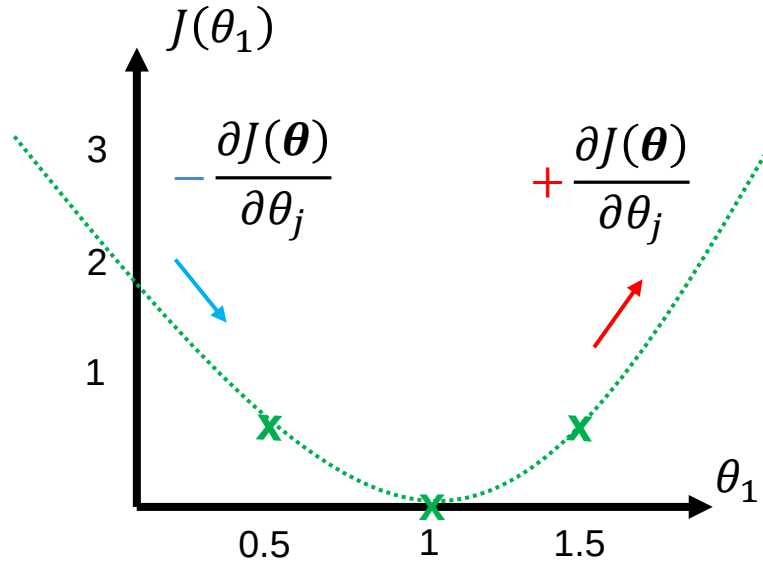


Where points a gradient?

$J(\theta)$ value increases as the θ value **increases**



Next, let's see how to compute and use a gradient





Gradient search

Minimization criteria for linear regression (1/2)

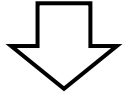
- By definition, $J(\boldsymbol{\theta})$ is minimized when $\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} = 0$:

$$\begin{aligned}\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}]^2 \\ &= 2 \frac{1}{2 \cdot 1} \sum_{i=1}^m [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}] \frac{\partial}{\partial \theta_j} \sum_{i=1}^m [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}]\end{aligned}$$

- Let's assume $m = 1$ and multivariate model!
- **Note:** Since the loss function is quadratic, the partial derivative will be a linear function.

Minimization criteria for linear regression (2/2)

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} \Big|_{\boldsymbol{\theta}} \stackrel{(m=1)}{=} [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}] \frac{\partial}{\partial \theta_j} [\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots + \theta_n x_n^{(i)} - y^{(i)}]$$



$$\frac{\partial J(\theta)}{\partial \theta_j} = [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}] x_j^{(i)}$$

How to compute gradient?

$$\frac{\partial J(\theta)}{\partial \theta_j} = \underbrace{[h_{\theta}(x^{(i)}) - y^{(i)}]}_{\text{Hypothesis}} \underbrace{x_j^{(i)}}_{\text{Training data set}}$$

Hypothesis
(e.g., $h_{\theta}(x) = 4 + 2.1 x$)

Training data set
 $(x_j^{(i)}, y^{(i)})$

- Now, we assume univariate linear model and compute the gradient as follows:

Example: Gradient descent for univariate model

$$\left. \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} \right|_{\theta_0, \theta_1} \stackrel{(m=1)}{=} [h_{\theta}(x^{(i)}) - y^{(i)}] \frac{\partial}{\partial \theta_j} [h_{\theta}(x^{(i)}) - y^{(i)}]$$



$$\begin{cases} \frac{\partial J(\theta)}{\partial \theta_0} = h_{\theta}(x^{(i)}) - y^{(i)} \\ \frac{\partial J(\theta)}{\partial \theta_1} = [h_{\theta}(x^{(i)}) - y^{(i)}] x_1^{(i)} \end{cases}$$

$$h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)}$$

Example: Gradient descent for univariate model

- The loss function is a quadratic function, so the partial derivative will be a linear function

$$\left. \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} \right|_{\theta_0, \theta_1} \stackrel{(m=1)}{=} [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}] \frac{\partial}{\partial \theta_j} [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}]$$



$$\begin{cases} \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0} = \boxed{h_{\boldsymbol{\theta}}(x^{(i)})} - y^{(i)} \\ \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_1} = [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}] x_1^{(i)} \end{cases}$$

Many forms of learning require updating parameters to minimize a loss

$$h_{\boldsymbol{\theta}}(x^{(i)}) = \boxed{\theta_0 + \theta_1 x_1^{(i)}}$$

Gradient descent parameter update rule

$\theta \leftarrow$ select any point in the parameter space

loop until convergence

for each θ_j in θ **do**

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

Until convergence to the minimum possible loss

$$\begin{cases} \frac{\partial J(\theta)}{\partial \theta_0} = h_{\theta}(x^{(i)}) - y^{(i)} \\ \frac{\partial J(\theta)}{\partial \theta_1} = [h_{\theta}(x^{(i)}) - y^{(i)}]x_1^{(i)} \end{cases}$$

- The parameter α is called the **step size** or **learning rate** when we are minimizing loss
 - It can be **fixed**, or it can **decay over time** as the learning process proceeds

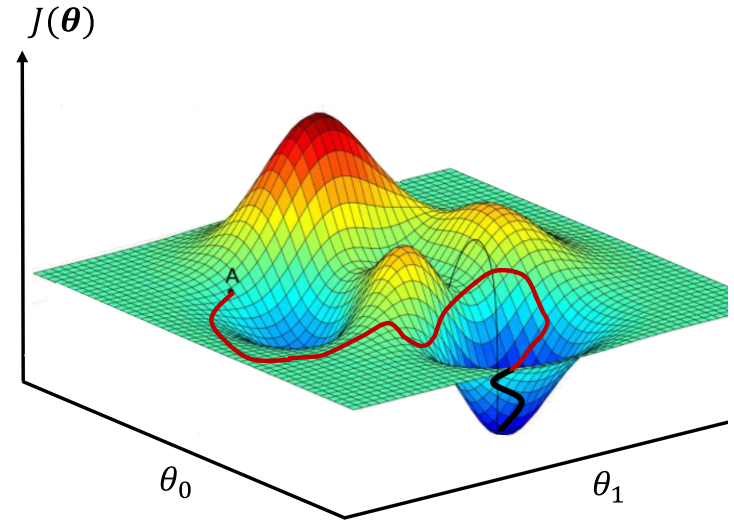


Update rule interpretation

Interpretation of the parameter rule update

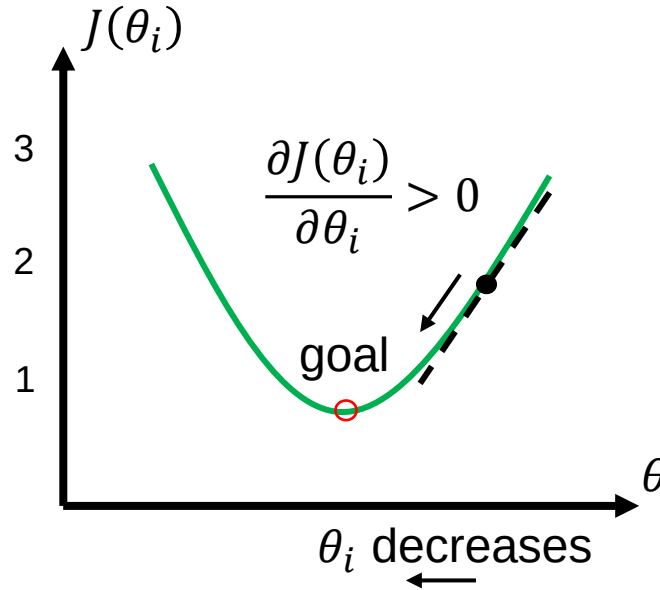
- Start from a random point (θ_0, θ_1)
- Go downhill by changing (θ_0, θ_1)
 - θ_j s are updated by subtracting gradient from current value of θ_j

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$



Gradient-based update rule (1/2)

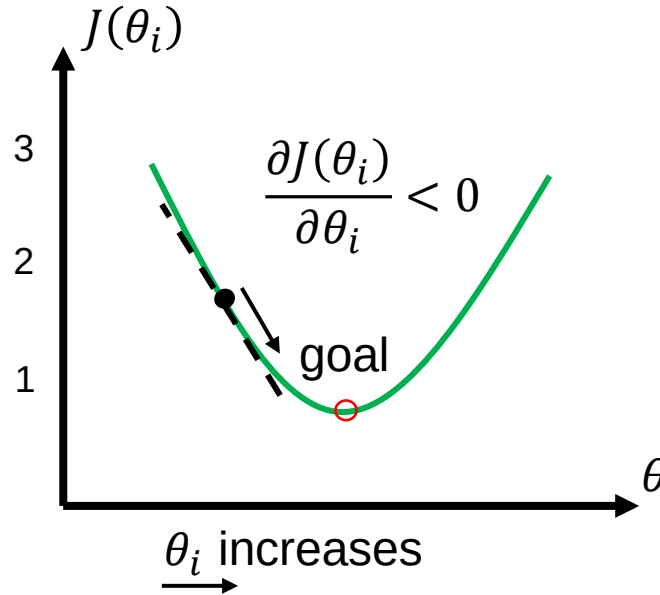
$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial J(\theta_i)}{\partial \theta_i}$$



$$\theta_i \leftarrow \theta_i - \alpha \times (+)$$

Gradient-based update rule (2/2)

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial J(\theta_i)}{\partial \theta_i}$$



$$\theta_i \leftarrow \theta_i - \alpha \times (-)$$

General parameter update for the linear regression

- We obtain the **general** parameter learning rule by plugging $\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}$ into

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}$$

as:

$$\theta_j \leftarrow \theta_j - \alpha \underbrace{[h_{\theta}(x^{(i)}) - y^{(i)}]}_{\text{Gradient}} x_j^{(i)}$$

- **Note:** As we approach local minimum, the gradient is approaching zero, thus the parameter taken steps are smaller and smaller

Parameter update rule for univariate model

- When plugging the gradient $\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}$ of univariate regression model into
$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}$$

we obtain the parameter learning rule:

$$\begin{cases} \theta_0 \leftarrow \theta_0 - \alpha [h_{\theta}(x^{(i)}) - y^{(i)}] \\ \theta_1 \leftarrow \theta_1 - \alpha [h_{\theta}(x^{(i)}) - y^{(i)}] x_1^{(i)} \end{cases}$$

Remark 1: Gradient updates

- The θ_j updates make intuitive sense

$$\begin{cases} \theta_0 \leftarrow \theta_0 - \alpha [h_{\theta}(x^{(i)}) - y^{(i)}] \\ \theta_1 \leftarrow \theta_1 - \alpha [h_{\theta}(x^{(i)}) - y^{(i)}] x_1^{(i)} \end{cases}$$

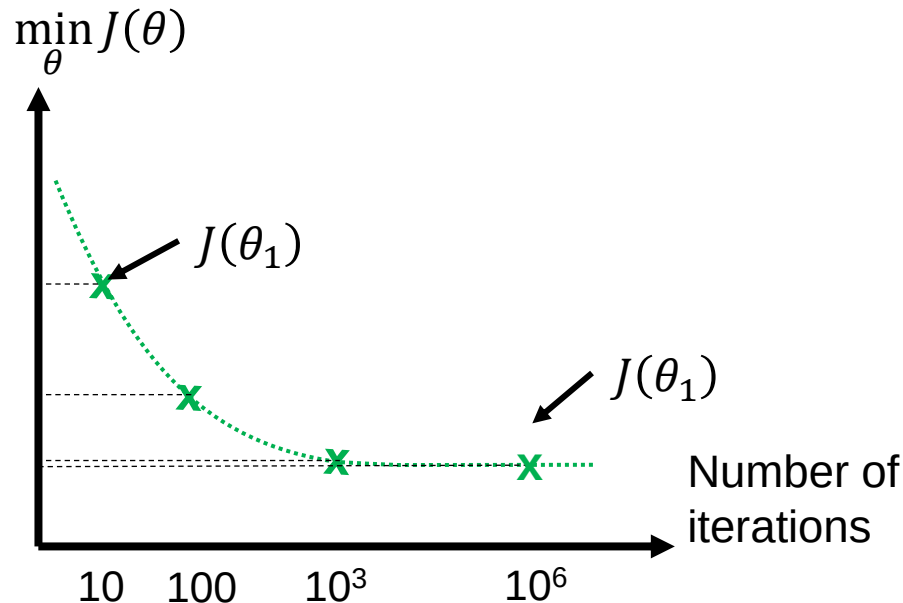
- if $h_{\theta}(x) > y$ the output of hypothesis is too large, leading to reduction of θ_0

AND

- reducing θ_1 if x was a positive input or
- increasing θ_1 if x was a negative input

Remark 2: How to test gradient descent algorithm?

- Check the decrease of the minimum value of the cost function $J(\theta)$
 - Convergence is achieved if/when $J(\theta)$ decreases by **less than ε** (e.g., 10^{-6}) in **a single iteration**

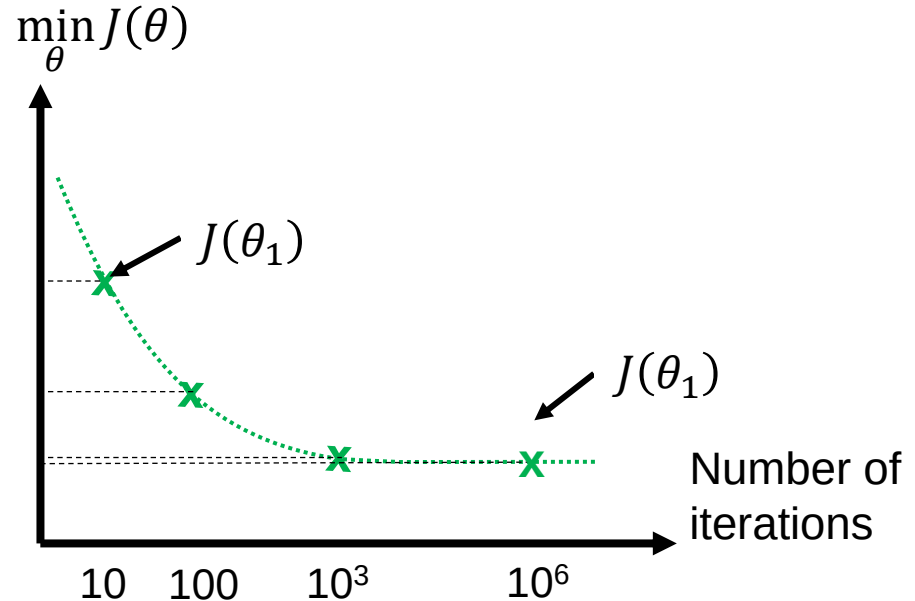


Remark 2: How to test gradient descent algorithm?

- Check the decrease of the minimum value of the cost function $J(\theta)$
 - Convergence is achieved if/when $J(\theta)$ decreases by **less than ε** (e.g., 10^{-6}) in **a single iteration**

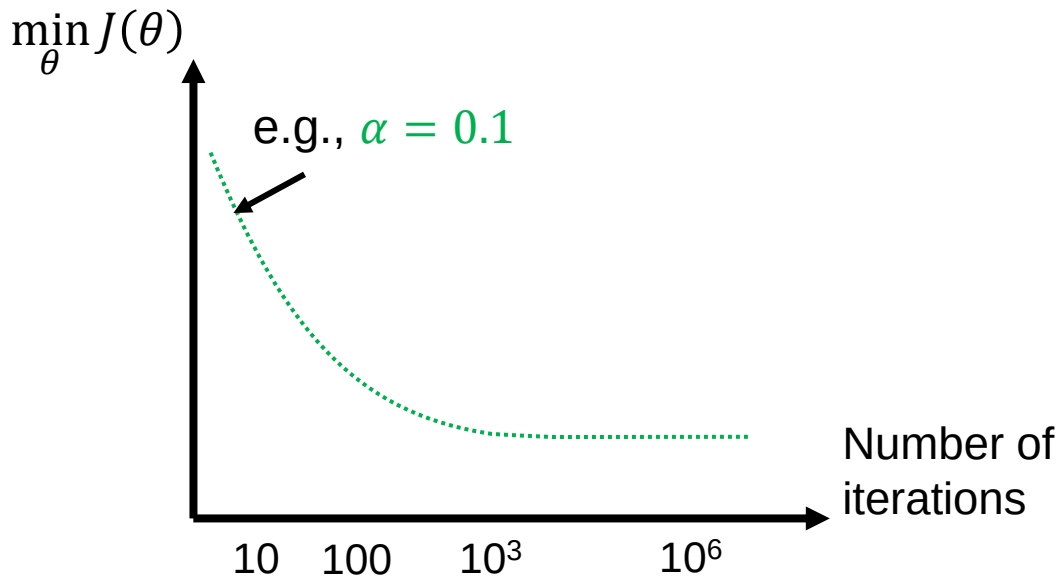
- Following this test, we can select the step size α to decrease $J(\theta)$ in each iteration

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial J(\theta_i)}{\partial \theta_i}$$



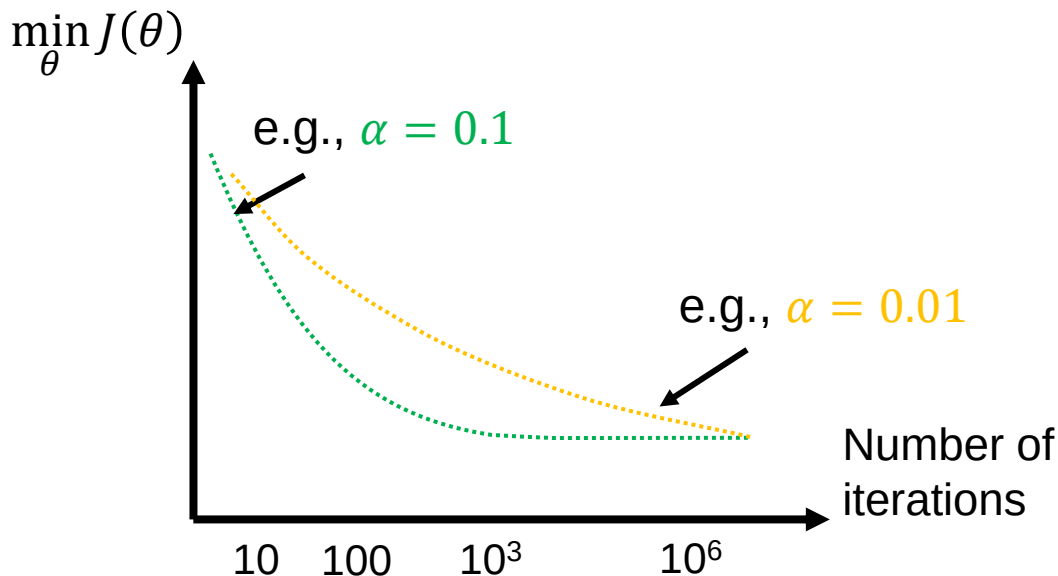
Remark 3: Selection of step size

- In principle, for a small α , $J(\theta)$ should decrease on every iteration
 - For very small α , convergence is slow



Remark 3: Selection of step size

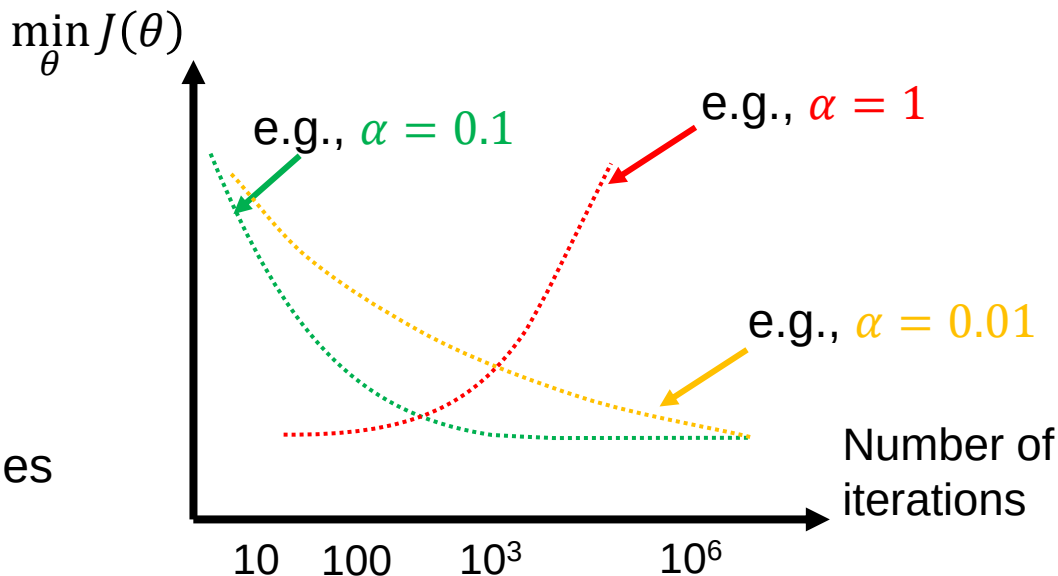
- In principle, for a small α , $J(\theta)$ should decrease on every iteration
 - For sufficiently small α , $J(\theta)$ decreases on each iteration



Remark 3: Selection of step size

- In principle, for a small α , $J(\theta)$ should decrease on every iteration
 - For large α , convergence is not guaranteed and $J(\theta)$ may not decrease for each iteration

The selection of α is done by experimenting with different values





Other gradient algorithms

Batch gradient descent algorithm

- The algorithm utilizes all m training examples

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}] x_j^{(i)}$$

- For small learning rate, the convergence to the unique global minimum is guaranteed
- But, small learning rate may lead to slow convergence
- Problem with batch gradient descent is long execution time!

Stochastic gradient descent algorithm

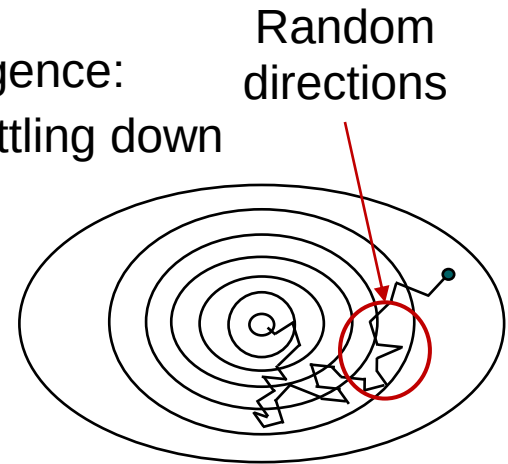
- Considers only a single training point at a time:

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = [h_{\theta}(x^{(i)}) - y^{(i)}] \frac{\partial}{\partial \theta_j} (h_{\theta}(x^{(i)}) - y^{(i)})$$

- Takes a step after each gradient computation (per training example)

$$\theta_j \leftarrow \theta_j - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- However, a fixed learning rate does not guarantee convergence:
 - algorithm can oscillate around the minimum without settling down



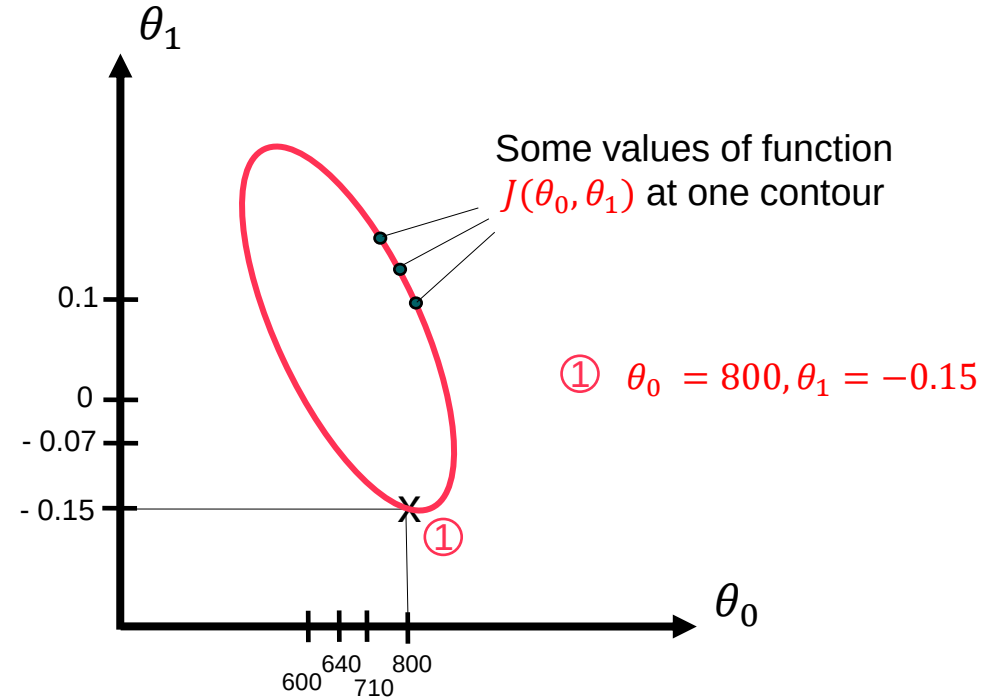
Remarks

- We have to cycle through all the training data for every step, and there may be many steps 😊
- To calculate j^{th} parameter θ_j , entire training set ($\sum_{i=1}^m$) is utilized
 - α is small and steps in convergence are small it takes a lot of time to converge
 - α is large and steps are large so steepest descent may end up overshooting the minimum if steps are too large
- The gradient of a function shows the direction of steepest descent of the function
- Batch Gradient Descent algorithm has drawback in the case of large training set
 - E.g., for large m a large amount of computation is needed before you perform a single step
- Stochastic gradient descent is faster than batch gradient descent

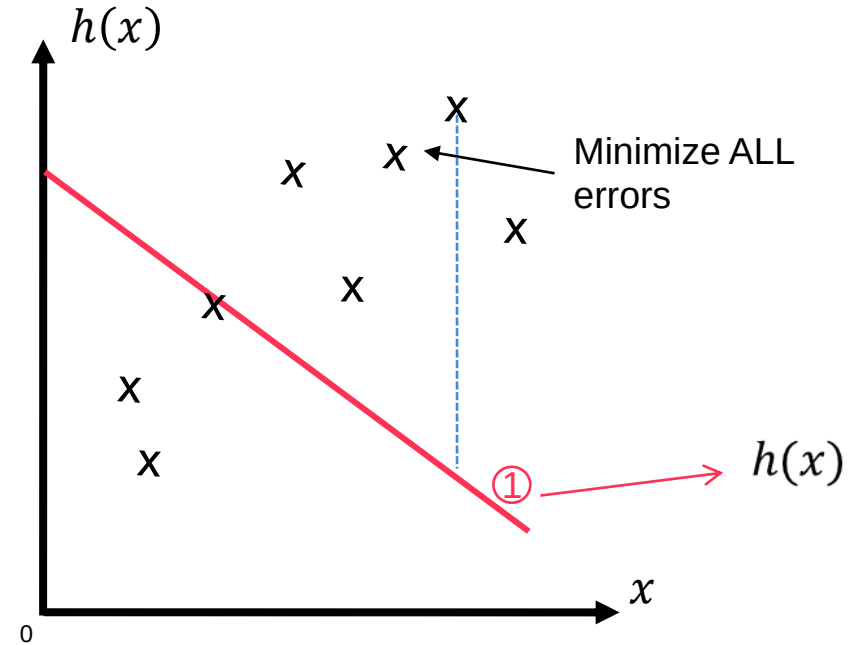
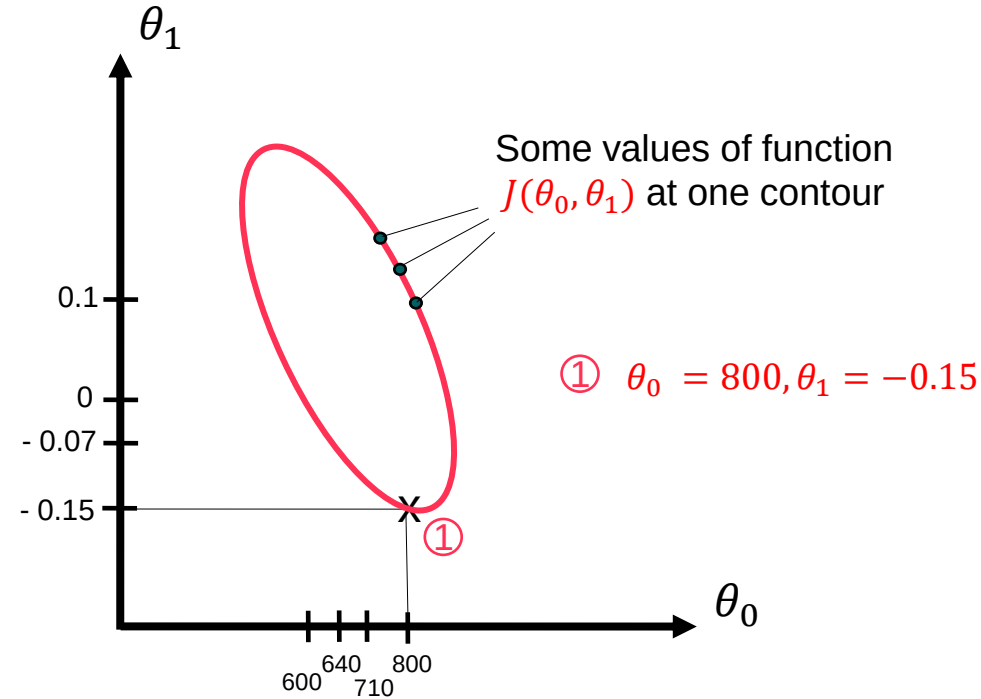


Understanding the weight and hypothesis space

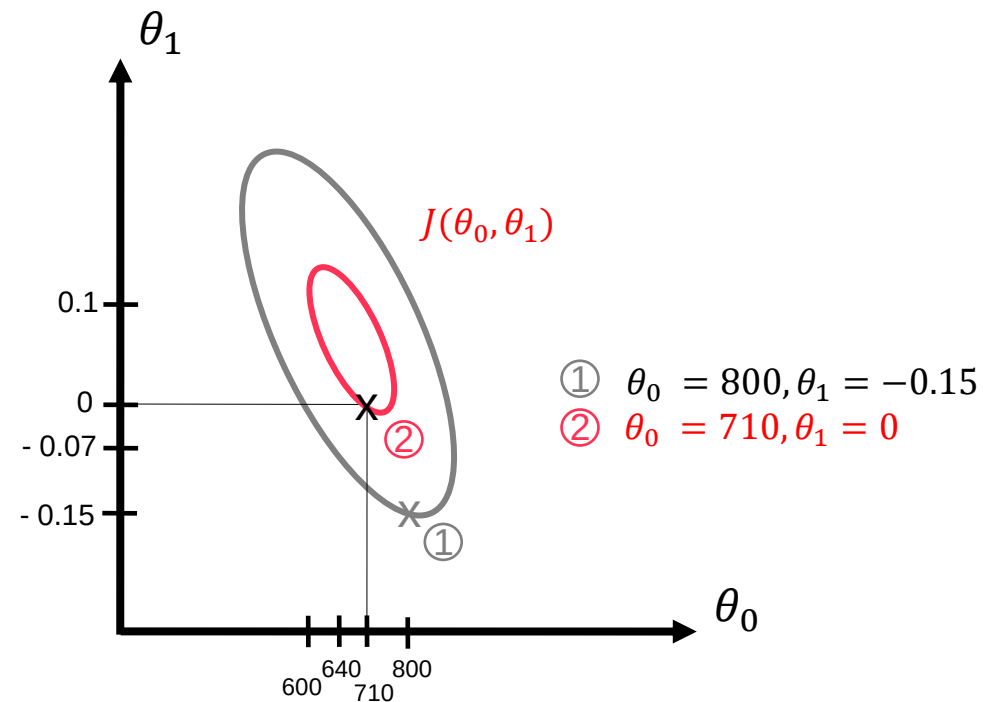
Relationship between $h(x)$ and $J(\cdot)$



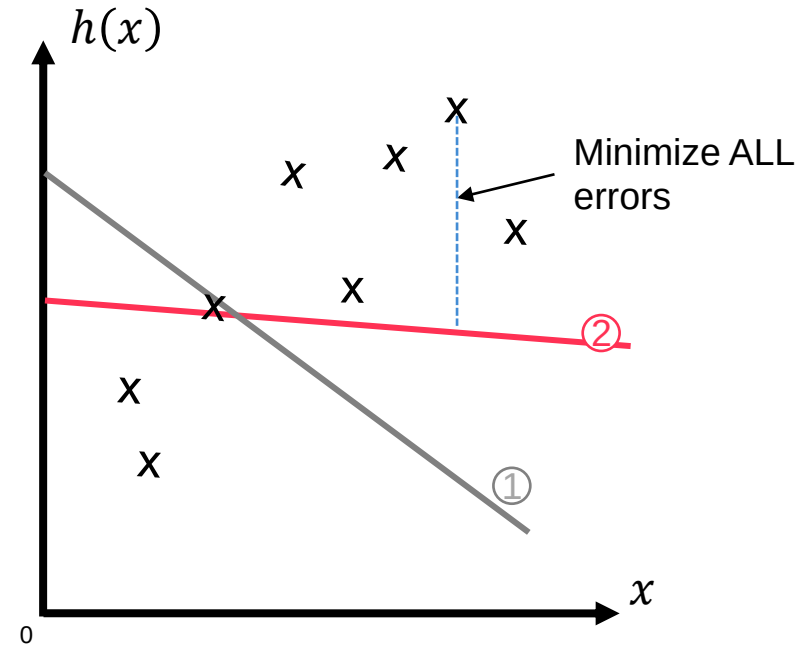
Relationship between $h(x)$ and $J(\cdot)$



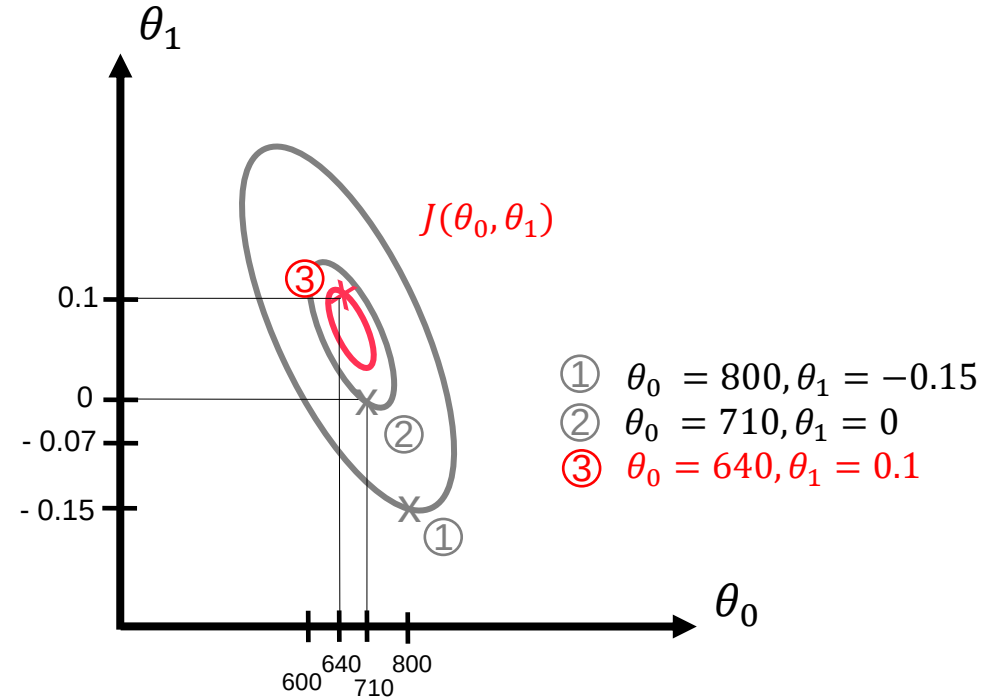
Relationship between $h(x)$ and $J(\cdot)$



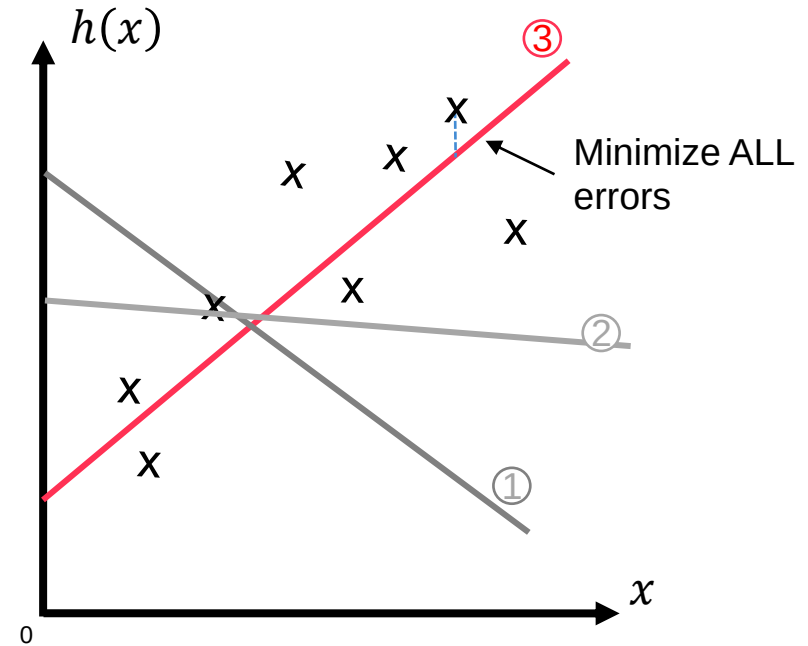
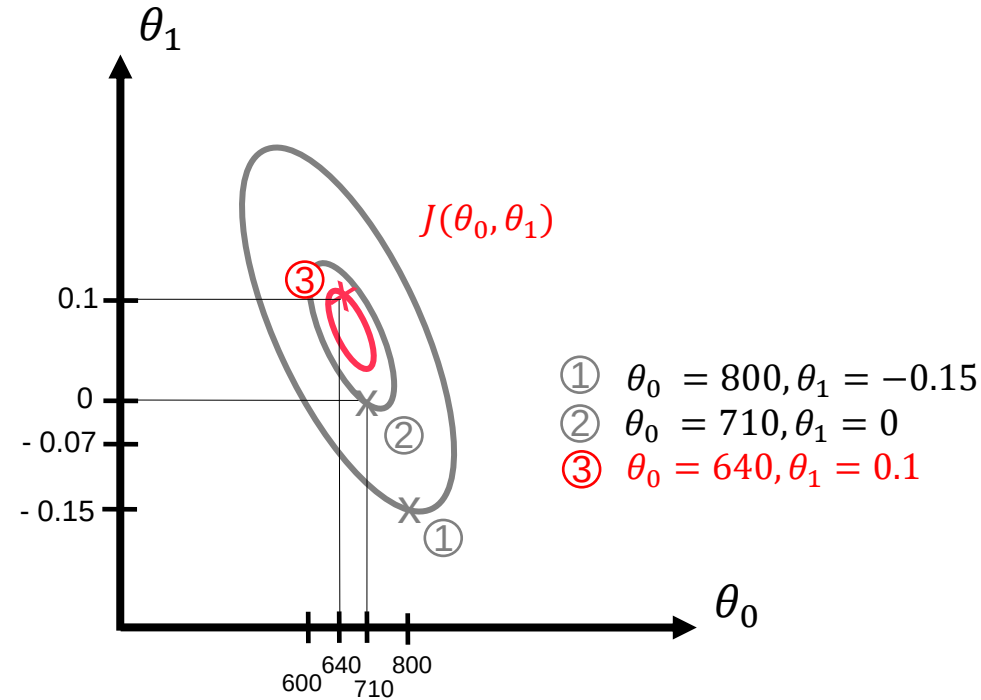
Relationship between $h(x)$ and $J(\cdot)$



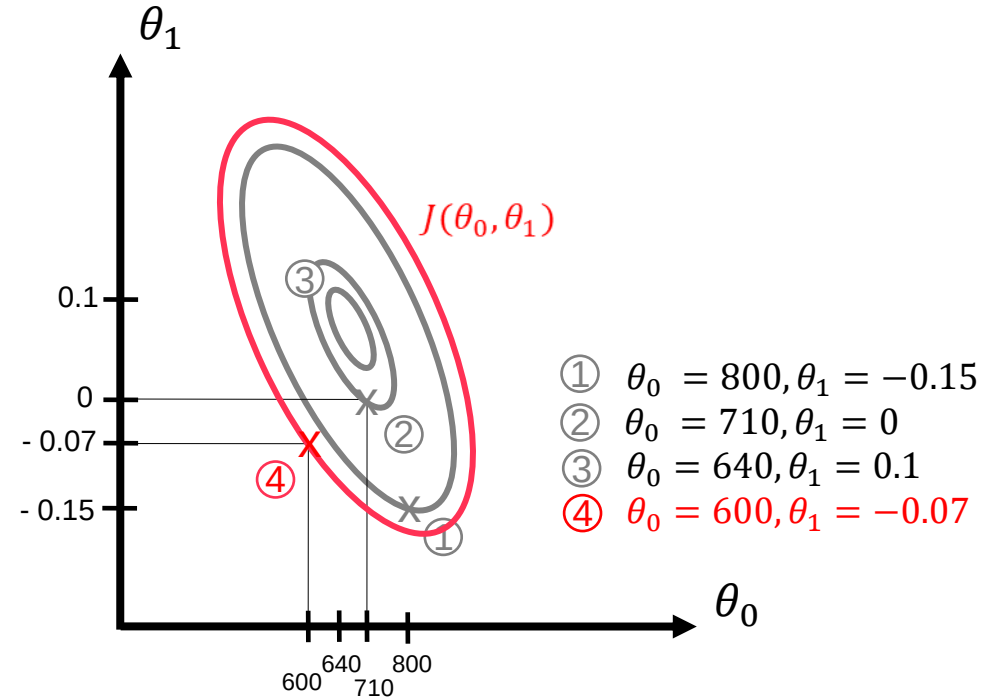
Relationship between $h(x)$ and $J(\cdot)$



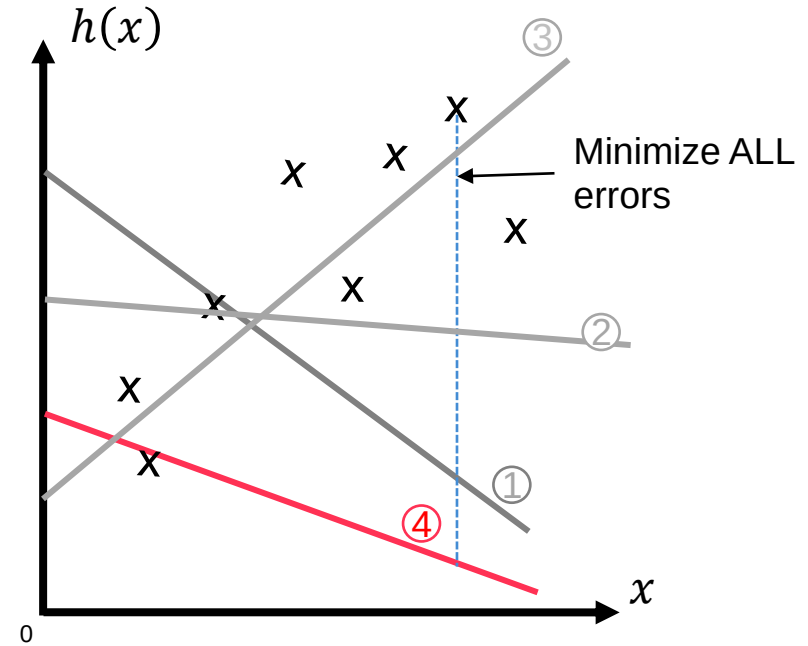
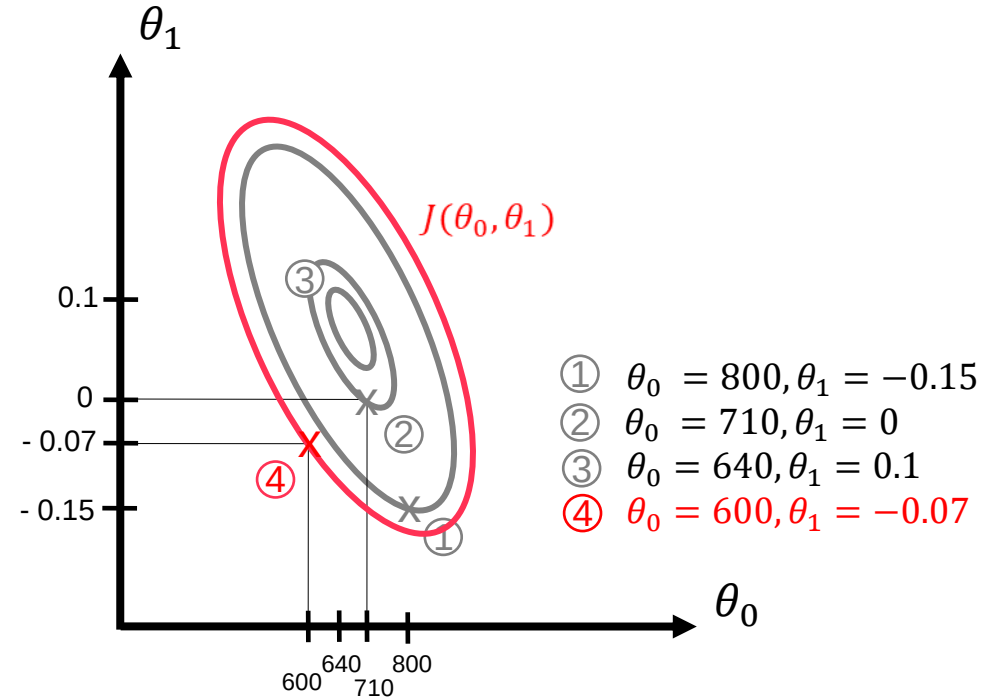
Relationship between $h(x)$ and $J(\cdot)$



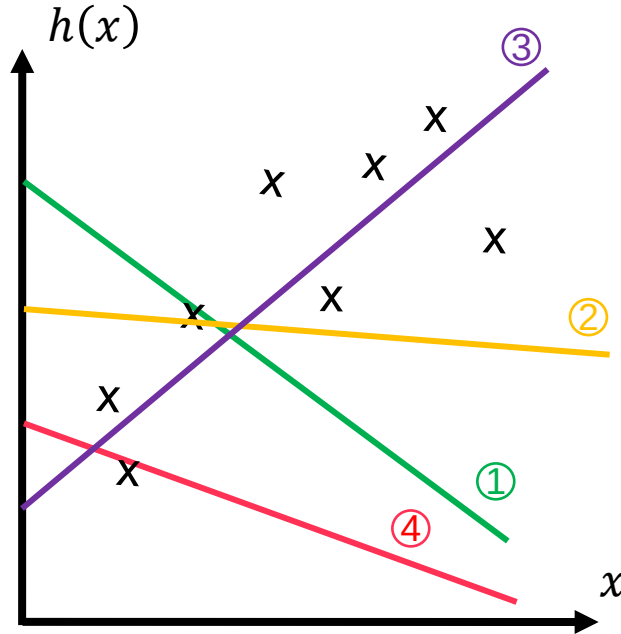
Relationship between $h(x)$ and $J(\cdot)$



Relationship between $h(x)$ and $J(\cdot)$



Hypothesis space $\mathcal{H}$



Many hypotheses

① ② ③ ④ ...



Locally weighted regression

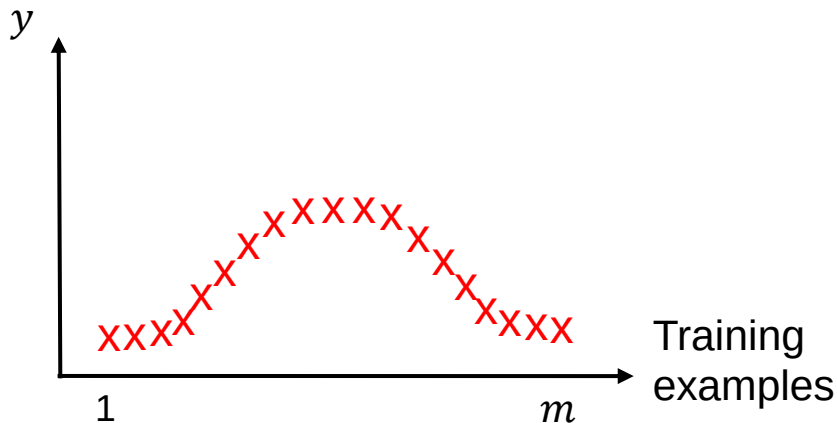
Chapter 18.8.4

Batch linear regression and training examples

- Evaluate $h_{\theta}(x)$ at a certain position x
- We fit θ to minimize the cost

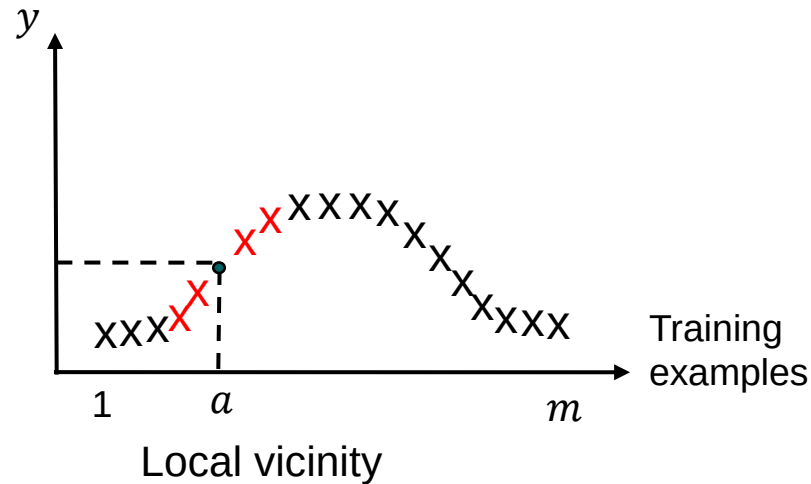
Linear regression considers all m training samples to return a fit h_{θ}

$$\sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)}))^2$$



Principle of locally weighted (linear) regression

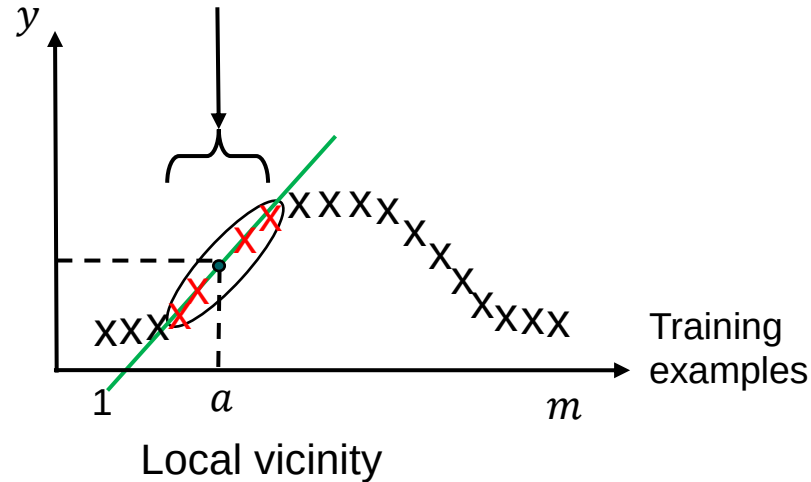
- In contrast, local linear regression looks only at local points in the vicinity of a



Principle of locally weighted (linear) regression

- In contrast, local linear regression looks only at local points in the vicinity of a

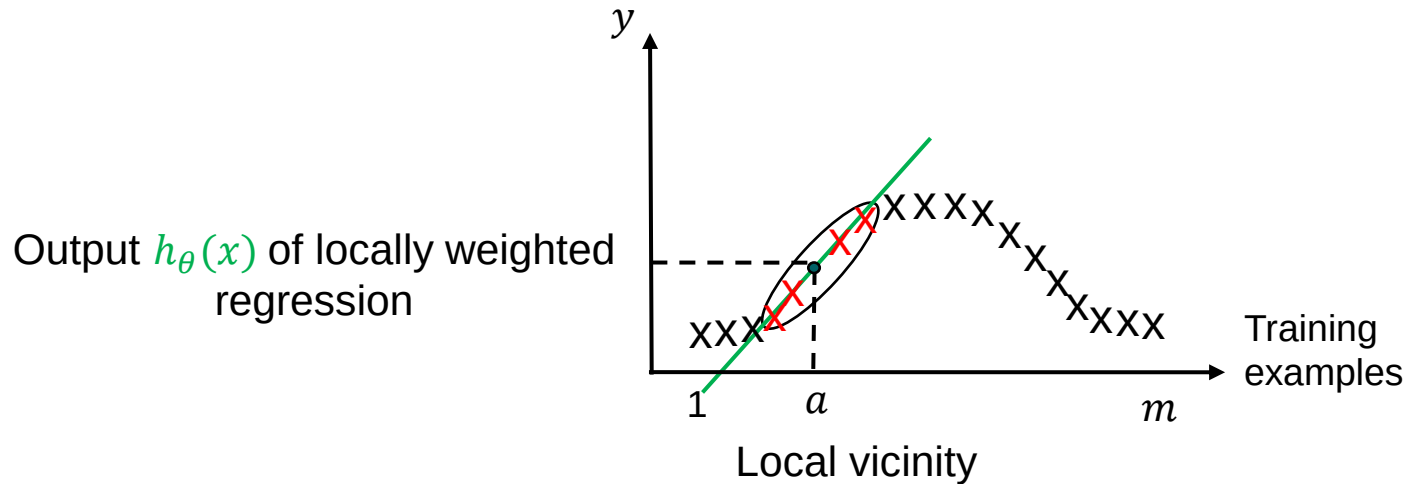
We take a limited set of (red) examples and apply regression to fit the model $h_{\theta}(x)$



Principle of locally weighted (linear) regression

- In contrast, local linear regression looks only at local points in the vicinity of a

We take a limited set of (red) examples and apply regression to fit the model $h_{\theta}(x)$

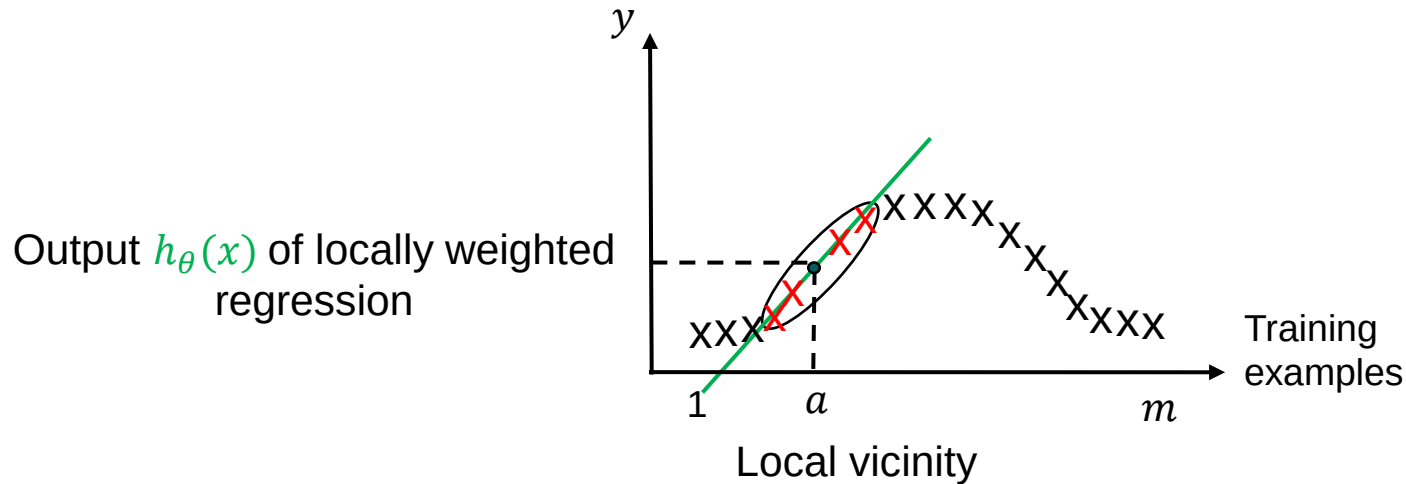


Principle of locally weighted (linear) regression

- In contrast, local linear regression looks only at local points in the vicinity of a

We take a limited set of (red) examples and apply

The model changes depending on where you evaluate it.



Weight selection example

- Locally weighted regression fits θ to minimize

$$\sum_{i=1}^m w^{(i)} [y^{(i)} - \theta^T x^{(i)}]^2$$

Weight selection example

- Locally weighted regression fits θ to minimize

$$\sum_{i=1}^m w^{(i)} [y^{(i)} - \theta^T x^{(i)}]^2$$

- With one example of the weight being:

$$w^{(i)} = e^{-\frac{(x^{(i)} - a)^2}{2}}$$

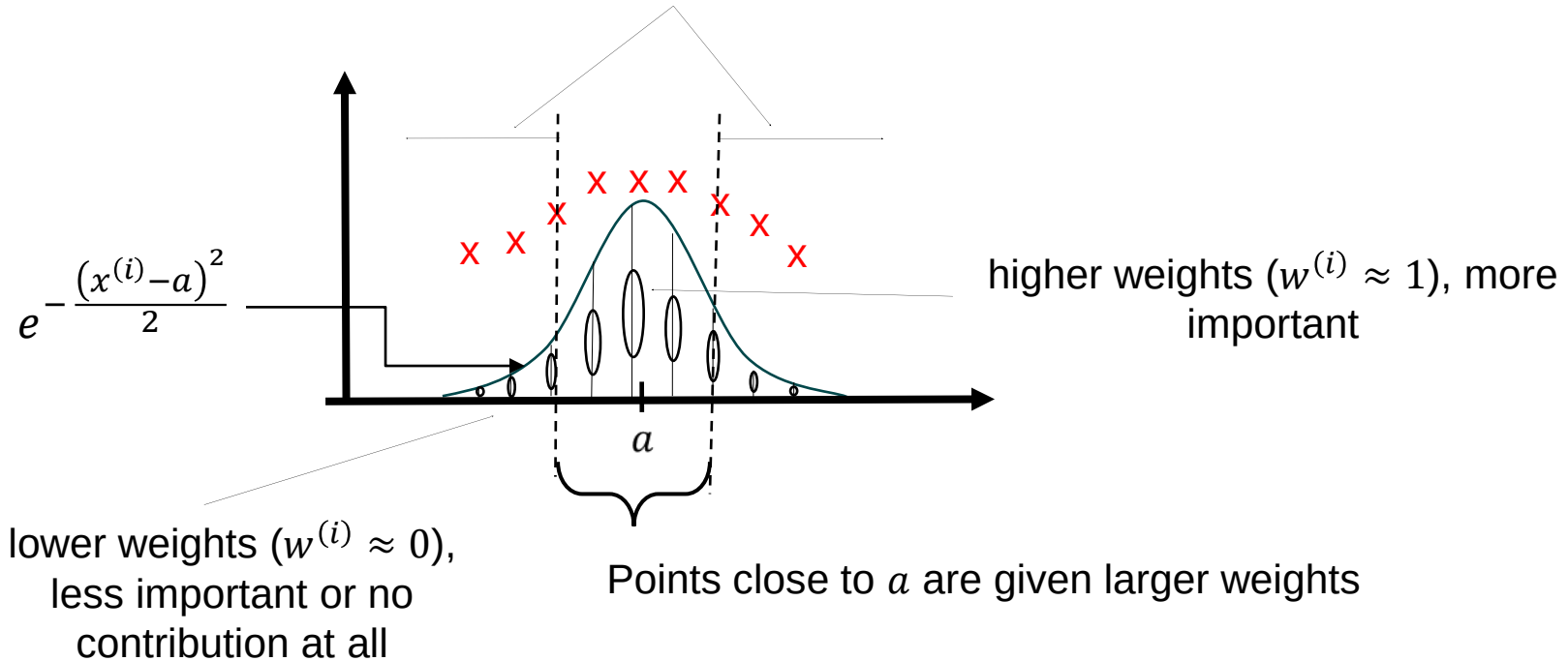
a bell shape curve
centered around a

- Conditions:

- $|x^{(i)} - a| \ll 1, w^{(i)} \approx 1$
- $|x^{(i)} - a| \gg 1, w^{(i)} \approx 0$

Illustration of weighted linear regression

Points away from a are given small weights

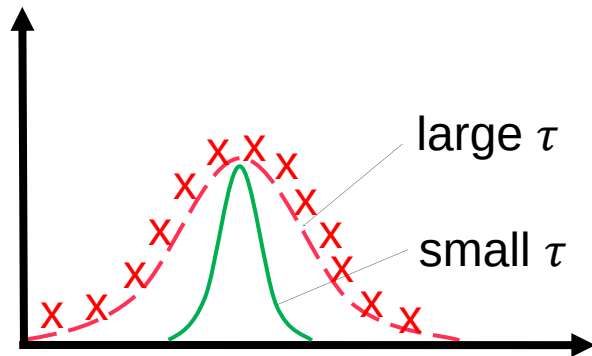


Remarks: Locally weighted regression

- This is non-parametric learning algorithm, so no building of model is done since we need full set of examples to make prediction
- New design parameter “bandwidth τ ”

$$w^{(i)} = e^{-\frac{(x^{(i)}-a)^2}{2\tau^2}}$$

- Bandwidth τ controls how fast the weights fall down with the distance from a
- **Linear regression**: one fixed model for all data!
- **Locally weighted linear regression**: different local models for different regions!



Thank you!

